

Topic:

Menu-Driven C Program to Verify the Left and Right Distributive Properties of Matrix Multiplication over Matrix Addition using Dynamically Allocated 2D Arrays

Name	Rudra Pratap Singh
SAP ID	590032811
Batch	16

1. Objective

To design and implement a menu-driven C program that reads variable-length (dynamically sized) two-dimensional arrays representing matrices, and verifies the Left Distributive Property, $A \times (B + C) = (A \times B) + (A \times C)$, and the Right Distributive Property, $(A + B) \times C = (A \times C) + (B \times C)$, of matrix multiplication over matrix addition, using separate, well-defined functions for each operation.

2. Theory

Matrix multiplication is distributive over matrix addition, provided the dimensions of the matrices are compatible for the operations involved.

Left Distributive Property

For a matrix A of order $(m \times n)$ and matrices B, C of order $(n \times p)$:

$$A \times (B + C) = (A \times B) + (A \times C)$$

Here, $(B + C)$ is first computed $(n \times p)$, and then pre-multiplied by A, giving a result of order $(m \times p)$ on the left-hand side (LHS). On the right-hand side (RHS), A is multiplied separately with B and with C, and the two $(m \times p)$ products are added.

Right Distributive Property

For matrices A, B of order $(m \times n)$ and matrix C of order $(n \times p)$:

$$(A + B) \times C = (A \times C) + (B \times C)$$

Here, $(A + B)$ is first computed $(m \times n)$, and then post-multiplied by C, giving a result of order $(m \times p)$ on the LHS. On the RHS, A and B are individually multiplied with C, and the two $(m \times p)$ products are added.

In both cases, if every corresponding element of the LHS and RHS resultant matrices is equal, the property is said to be verified for the given input matrices.

3. Algorithm

- Start the program and display the menu with three choices: (1) Left Distributive Property, (2) Right Distributive Property, and (3) Exit.
- Read the user's choice.
- If choice = 1: Read dimensions m, n, p. Dynamically create matrices A $(m \times n)$, B $(n \times p)$ and C $(n \times p)$ and read their elements.
- Compute LHS = $A \times (B + C)$ using the addMatrix() and multiplyMatrix() functions.
- Compute RHS = $(A \times B) + (A \times C)$ using the multiplyMatrix() and addMatrix() functions.
- Display matrices A, B, C, LHS and RHS. Compare LHS and RHS element-wise using isEqualMatrix(); print whether the Left Distributive Property is verified.

- If choice = 2: Read dimensions m, n, p. Dynamically create matrices A ($m \times n$), B ($m \times n$) and C ($n \times p$) and read their elements.
- Compute $LHS = (A + B) \times C$ and $RHS = (A \times C) + (B \times C)$; display all matrices and state whether the Right Distributive Property is verified.
- If choice = 3: Terminate the program.
- Free all dynamically allocated memory after each operation and repeat the menu until the user chooses to exit.

4. Functions Used

Function	Purpose
<code>createMatrix()</code>	Dynamically allocates memory (<code>int **</code>) for a matrix of given rows $\times$ cols.
<code>readMatrix()</code>	Reads all elements of a matrix from the user, row by row.
<code>printMatrix()</code>	Displays a matrix in a neatly formatted, labelled, tabular layout.
<code>addMatrix()</code>	Returns the element-wise sum of two matrices of identical dimension.
<code>multiplyMatrix()</code>	Returns the matrix product of two compatible matrices ($r1 \times c1$) and ($c1 \times c2$).
<code>isEqualMatrix()</code>	Compares two matrices element-wise; returns 1 if equal, 0 otherwise.
<code>freeMatrix()</code>	Releases all dynamically allocated memory of a matrix to avoid memory leaks.
<code>leftDistributive()</code>	Reads A, B, C; computes and compares $A \times (B + C)$ with $(A \times B) + (A \times C)$.
<code>rightDistributive()</code>	Reads A, B, C; computes and compares $(A + B) \times C$ with $(A \times C) + (B \times C)$.
<code>main()</code>	Displays the menu in a loop and invokes the appropriate function based on user choice.

5. Source Code

File name: distributive_property.c

```

/*
 * =====
 * Program Title : Verification of Left and Right Distributive Properties
 *                 of Matrix Multiplication over Matrix Addition
 *
 * Left  Distributive Property :  $A * (B + C) = (A * B) + (A * C)$ 
 * Right Distributive Property :  $(A + B) * C = (A * C) + (B * C)$ 
 *
 * Description : This is a menu-driven C program that reads variable
 *               length (dynamically sized) 2D arrays (matrices), from
 *               the user, performs the required matrix addition and
 *               multiplication operations using separate functions, and
 *               verifies whether the chosen distributive property holds.
 *
 * Name       : Rudra Pratap Singh
 * SAP ID     : 590032811
 * Batch      : 16
 * =====
 */

#include <stdio.h>
#include <stdlib.h>

/* -----
 * Function      : createMatrix
 * Purpose       : Dynamically allocates memory for a matrix of size
 *                rows x cols
 * Parameters    : rows, cols - dimensions of the matrix
 * Returns       : pointer to the dynamically allocated matrix (int **)
 * ----- */
int **createMatrix(int rows, int cols)
{
    int **mat = (int **)malloc(rows * sizeof(int *));
    if (mat == NULL)
    {
        printf("Memory allocation failed!\n");
        exit(1);
    }

    for (int i = 0; i < rows; i++)
    {
        mat[i] = (int *)malloc(cols * sizeof(int));
        if (mat[i] == NULL)
        {
            printf("Memory allocation failed!\n");
            exit(1);
        }
    }
    return mat;
}

/* -----
 * Function      : freeMatrix
 * Purpose       : Frees the dynamically allocated memory of a matrix
 * Parameters    : mat - matrix to be freed, rows - number of rows
 * Returns       : void
 * ----- */
void freeMatrix(int **mat, int rows)
{
    for (int i = 0; i < rows; i++)
        free(mat[i]);
    free(mat);
}

```

```

}

/* -----
 * Function      : readMatrix
 * Purpose       : Reads elements of a matrix of size rows x cols from
 *                the user
 * Parameters    : mat - matrix to be filled, rows, cols - dimensions,
 *                name - character used to label the matrix on screen
 * Returns       : void
 * ----- */
void readMatrix(int **mat, int rows, int cols, char name)
{
    printf("\nEnter elements of matrix %c (%d x %d row-wise):\n", name, rows, cols);
    for (int i = 0; i < rows; i++)
    {
        for (int j = 0; j < cols; j++)
        {
            printf("%c[%d][%d] = ", name, i + 1, j + 1);
            scanf("%d", &mat[i][j]);
        }
    }
}

/* -----
 * Function      : printMatrix
 * Purpose       : Displays a matrix of size rows x cols in proper
 *                tabular / row-column format
 * Parameters    : mat - matrix to display, rows, cols - dimensions,
 *                label - string label to print above the matrix
 * Returns       : void
 * ----- */
void printMatrix(int **mat, int rows, int cols, const char *label)
{
    printf("\n%s (%d x %d):\n", label, rows, cols);
    for (int i = 0; i < rows; i++)
    {
        printf("\t");
        for (int j = 0; j < cols; j++)
            printf("%6d ", mat[i][j]);
        printf("\n");
    }
}

/* -----
 * Function      : addMatrix
 * Purpose       : Adds two matrices of the same dimension (rows x cols)
 * Parameters    : A, B - input matrices, rows, cols - dimensions
 * Returns       : pointer to the resultant sum matrix (int **)
 * ----- */
int **addMatrix(int **A, int **B, int rows, int cols)
{
    int **result = createMatrix(rows, cols);
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
            result[i][j] = A[i][j] + B[i][j];
    return result;
}

/* -----
 * Function      : multiplyMatrix
 * Purpose       : Multiplies two matrices A (r1 x c1) and B (c1 x c2)
 * Parameters    : A, B - input matrices, r1, c1, c2 - dimensions
 * Returns       : pointer to the resultant product matrix (r1 x c2)
 * ----- */
int **multiplyMatrix(int **A, int **B, int r1, int c1, int c2)
{
    int **result = createMatrix(r1, c2);
    for (int i = 0; i < r1; i++)
    {
        for (int j = 0; j < c2; j++)
        {

```

```

        result[i][j] = 0;
        for (int k = 0; k < c1; k++)
            result[i][j] += A[i][k] * B[k][j];
    }
}
return result;
}

/* -----
* Function      : isEqualMatrix
* Purpose       : Compares two matrices of the same dimension for
*                equality (used to verify distributive property)
* Parameters    : A, B - matrices to compare, rows, cols - dimensions
* Returns       : 1 if matrices are equal, 0 otherwise
* ----- */
int isEqualMatrix(int **A, int **B, int rows, int cols)
{
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
            if (A[i][j] != B[i][j])
                return 0;
    return 1;
}

/* -----
* Function      : leftDistributive
* Purpose       : Reads matrices A(m x n), B(n x p), C(n x p), computes
*                LHS = A*(B+C) and RHS = (A*B)+(A*C), displays both
*                and verifies the LEFT distributive property
* Parameters    : none
* Returns       : void
* ----- */
void leftDistributive(void)
{
    int m, n, p;

    printf("\n--- Left Distributive Property : A * (B + C) = (A * B) + (A * C) ---\n");
    printf("Matrix A is (m x n), Matrix B and C are (n x p)\n");
    printf("Enter m (rows of A): ");
    scanf("%d", &m);
    printf("Enter n (cols of A = rows of B = rows of C): ");
    scanf("%d", &n);
    printf("Enter p (cols of B = cols of C): ");
    scanf("%d", &p);

    int **A = createMatrix(m, n);
    int **B = createMatrix(n, p);
    int **C = createMatrix(n, p);

    readMatrix(A, m, n, 'A');
    readMatrix(B, n, p, 'B');
    readMatrix(C, n, p, 'C');

    /* LHS = A * (B + C) */
    int **BplusC = addMatrix(B, C, n, p);
    int **LHS = multiplyMatrix(A, BplusC, m, n, p);

    /* RHS = (A * B) + (A * C) */
    int **AB = multiplyMatrix(A, B, m, n, p);
    int **AC = multiplyMatrix(A, C, m, n, p);
    int **RHS = addMatrix(AB, AC, m, p);

    printMatrix(A, m, n, "Matrix A");
    printMatrix(B, n, p, "Matrix B");
    printMatrix(C, n, p, "Matrix C");

    printf("\n-----\n");
    printMatrix(LHS, m, p, "LHS = A * (B + C)");
    printMatrix(RHS, m, p, "RHS = (A * B) + (A * C)");
}

```

```

printf("-----\n");

if (isEqualMatrix(LHS, RHS, m, p))
    printf("\nResult: LHS = RHS => Left Distributive Property is VERIFIED.\n");
else
    printf("\nResult: LHS != RHS => Left Distributive Property is NOT verified.\n");

/* free all dynamically allocated memory */
freeMatrix(A, m);
freeMatrix(B, n);
freeMatrix(C, n);
freeMatrix(BplusC, n);
freeMatrix(LHS, m);
freeMatrix(AB, m);
freeMatrix(AC, m);
freeMatrix(RHS, m);
}

/* -----
 * Function      : rightDistributive
 * Purpose       : Reads matrices A(m x n), B(m x n), C(n x p), computes
 *                LHS = (A+B)*C and RHS = (A*C)+(B*C), displays both
 *                and verifies the RIGHT distributive property
 * Parameters    : none
 * Returns       : void
 * ----- */
void rightDistributive(void)
{
    int m, n, p;

    printf("\n--- Right Distributive Property : (A + B) * C = (A * C) + (B * C) ---\n");
    printf("Matrix A and B are (m x n), Matrix C is (n x p)\n");
    printf("Enter m (rows of A = rows of B): ");
    scanf("%d", &m);
    printf("Enter n (cols of A = cols of B = rows of C): ");
    scanf("%d", &n);
    printf("Enter p (cols of C): ");
    scanf("%d", &p);

    int **A = createMatrix(m, n);
    int **B = createMatrix(m, n);
    int **C = createMatrix(n, p);

    readMatrix(A, m, n, 'A');
    readMatrix(B, m, n, 'B');
    readMatrix(C, n, p, 'C');

    /* LHS = (A + B) * C */
    int **ApB = addMatrix(A, B, m, n);
    int **LHS = multiplyMatrix(ApB, C, m, n, p);

    /* RHS = (A * C) + (B * C) */
    int **AC = multiplyMatrix(A, C, m, n, p);
    int **BC = multiplyMatrix(B, C, m, n, p);
    int **RHS = addMatrix(AC, BC, m, p);

    printMatrix(A, m, n, "Matrix A");
    printMatrix(B, m, n, "Matrix B");
    printMatrix(C, n, p, "Matrix C");

    printf("\n-----\n");
    printMatrix(LHS, m, p, "LHS = (A + B) * C");
    printMatrix(RHS, m, p, "RHS = (A * C) + (B * C)");
    printf("-----\n");

    if (isEqualMatrix(LHS, RHS, m, p))
        printf("\nResult: LHS = RHS => Right Distributive Property is VERIFIED.\n");
    else

```

```

printf("\nResult: LHS != RHS => Right Distributive Property is NOT verified.\n");

/* free all dynamically allocated memory */
freeMatrix(A, m);
freeMatrix(B, m);
freeMatrix(C, n);
freeMatrix(ApB, m);
freeMatrix(LHS, m);
freeMatrix(AC, m);
freeMatrix(BC, m);
freeMatrix(RHS, m);
}

/* -----
* Function      : main
* Purpose       : Displays the menu and calls the appropriate function
*                based on user's choice, in a loop, until Exit is chosen
* ----- */
int main(void)
{
    int choice;

    printf("=====\n");
    printf("  VERIFICATION OF DISTRIBUTIVE PROPERTIES OF MATRICES\n");
    printf("=====\n");

    do
    {
        printf("\n----- MENU ----- \n");
        printf("1. Verify Left  Distributive Property :  $A*(B+C) = (A*B)+(A*C)$ \n");
        printf("2. Verify Right Distributive Property :  $(A+B)*C = (A*C)+(B*C)$ \n");
        printf("3. Exit\n");
        printf("----- \n");
        printf("Enter your choice (1-3): ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                leftDistributive();
                break;
            case 2:
                rightDistributive();
                break;
            case 3:
                printf("\nExiting program. Thank you!\n");
                break;
            default:
                printf("\nInvalid choice! Please enter a number between 1 and 3.\n");
        }
    } while (choice != 3);

    return 0;
}

```

6. Sample Output & Verification

Two sample runs are shown below — one for each menu option — confirming correct compilation, execution and verification of both distributive properties.

Sample Run 1: Left Distributive Property [$A*(B+C) = (A*B)+(A*C)$]

```
=====
VERIFICATION OF DISTRIBUTIVE PROPERTIES OF MATRICES
=====

----- MENU -----
1. Verify Left Distributive Property :  $A*(B+C) = (A*B)+(A*C)$ 
2. Verify Right Distributive Property :  $(A+B)*C = (A*C)+(B*C)$ 
3. Exit
-----
Enter your choice (1-3): 1

--- Left Distributive Property :  $A * (B + C) = (A * B) + (A * C)$  ---
Matrix A is (m x n), Matrix B and C are (n x p)
Enter m (rows of A): 2
Enter n (cols of A = rows of B = rows of C): 2
Enter p (cols of B = cols of C): 2

Enter elements of matrix A (2 x 2 row-wise):
A[1][1]=1 A[1][2]=2 A[2][1]=3 A[2][2]=4

Enter elements of matrix B (2 x 2 row-wise):
B[1][1]=1 B[1][2]=0 B[2][1]=0 B[2][2]=1

Enter elements of matrix C (2 x 2 row-wise):
C[1][1]=2 C[1][2]=1 C[2][1]=1 C[2][2]=3

Matrix A (2 x 2):
    1    2
    3    4

Matrix B (2 x 2):
    1    0
    0    1

Matrix C (2 x 2):
    2    1
    1    3

-----
LHS = A * (B + C) (2 x 2):
    5    9
   13   19

RHS = (A * B) + (A * C) (2 x 2):
    5    9
   13   19

-----
Result: LHS = RHS => Left Distributive Property is VERIFIED.
```

Sample Run 2: Right Distributive Property [$(A+B)*C = (A*C)+(B*C)$]

```
=====
VERIFICATION OF DISTRIBUTIVE PROPERTIES OF MATRICES
=====
```

```

----- MENU -----
1. Verify Left Distributive Property :  $A*(B+C) = (A*B)+(A*C)$ 
2. Verify Right Distributive Property :  $(A+B)*C = (A*C)+(B*C)$ 
3. Exit
-----
Enter your choice (1-3): 2

--- Right Distributive Property :  $(A + B) * C = (A * C) + (B * C)$  ---
Matrix A and B are (m x n), Matrix C is (n x p)
Enter m (rows of A = rows of B): 2
Enter n (cols of A = cols of B = rows of C): 3
Enter p (cols of C): 2

Enter elements of matrix A (2 x 3 row-wise):
A[1][1]=1 A[1][2]=2 A[1][3]=3 A[2][1]=4 A[2][2]=5 A[2][3]=6

Enter elements of matrix B (2 x 3 row-wise):
B[1][1]=1 B[1][2]=0 B[1][3]=1 B[2][1]=0 B[2][2]=1 B[2][3]=0

Enter elements of matrix C (3 x 2 row-wise):
C[1][1]=1 C[1][2]=2 C[2][1]=3 C[2][2]=4 C[3][1]=5 C[3][2]=6

Matrix A (2 x 3):
    1    2    3
    4    5    6

Matrix B (2 x 3):
    1    0    1
    0    1    0

Matrix C (3 x 2):
    1    2
    3    4
    5    6

-----
LHS =  $(A + B) * C$  (2 x 2):
    28    36
    52    68

RHS =  $(A * C) + (B * C)$  (2 x 2):
    28    36
    52    68

-----
Result: LHS = RHS => Right Distributive Property is VERIFIED.

----- MENU -----
1. Verify Left Distributive Property :  $A*(B+C) = (A*B)+(A*C)$ 
2. Verify Right Distributive Property :  $(A+B)*C = (A*C)+(B*C)$ 
3. Exit
-----
Enter your choice (1-3): 3

Exiting program. Thank you!

```

7. Conclusion

The menu-driven C program was successfully designed, implemented and executed using dynamically allocated (variable-length) 2D arrays. Separate, modular functions were used for matrix creation, input, display, addition, multiplication, equality-checking and memory de-allocation. Test runs with both square and rectangular matrices confirmed that the Left Distributive Property, $A \times (B + C) = (A \times B) + (A \times C)$, and the Right Distributive Property, $(A + B) \times C = (A \times C) + (B \times C)$, hold true, as the LHS and RHS resultant matrices were found to be identical in every case, thereby verifying both properties of matrix algebra.